

halfedge: Catmull-Clark 细分模块设计文档

src/subdiv/catmull_clark.rs

halfedge 库

2026-07-04

目录

1	模块定位	2
1.1	核心 API	2
1.2	依赖关系	2
2	算法	2
2.1	面点 (Face Point)	2
2.2	边点 (Edge Point)	2
2.3	顶点更新 (Vertex Point)	3
2.3.1	内部顶点	3
2.3.2	边界顶点	3
2.4	面分裂	3
2.5	规模变化	3
3	数据结构	3
3.1	顶点布局	3
3.2	边键去重	4
3.3	边邻接面点收集	4
4	流程图	4
5	顶点更新分支	5
6	与 Loop 细分的对比	5
7	复杂度	5
8	边界处理	5
8.1	边界边点	5
8.2	边界顶点	6
9	退化与健壮性	6

1 模块定位

`catmull_clark.rs` 在 `traversal.rs`、`io.rs` 之上叠加 **Catmull-Clark** 细分能力, 用于任意多边形网格的光滑细分。与 Loop 细分 (仅支持三角网格) 不同, Catmull-Clark 适用于三角形、四边形及任意 k 边形面, 且一次细分后所有面变为四边形 (输出时三角化为 2 个三角形以兼容 `MeshStorage` 的三角面约束)。

1.1 核心 API

API	语义
<code>catmull_clark_subdivide(&MeshStorage) -> MeshStorage</code>	对任意多边形网格执行一次 Catmull-Clark 细分

1.2 依赖关系

- `storage::MeshStorage`: 底层网格容器
- `traversal::{FaceHalfEdges, VertexRing, is_boundary_vertex, is_boundary_edge}`: 面环遍历、顶点邻域遍历、边界判定
- `io::build_mesh_from_vertices_and_faces`: 从顶点位置 + 三角面索引构建输出网格

2 算法

Catmull-Clark 细分是**逼近型**细分方案, 是双三次 B 样条曲面的精确推广。每次细分引入三类新顶点 (面点、边点、更新后的原顶点), 并将每个 k 边形面分裂为 k 个四边形。

2.1 面点 (Face Point)

每个面的顶点平均值:

$$fp = \frac{1}{k} \sum_{i=0}^{k-1} v_i$$

其中 k 为面的边数。每个面产生恰好 1 个面点。

2.2 边点 (Edge Point)

对每条无向边 $e = (v_0, v_1)$, 设其两侧邻接面点为 fp_L, fp_R :

$$ep = \begin{cases} \frac{v_0 + v_1 + fp_L + fp_R}{4} & \text{内部边} \\ \frac{v_0 + v_1}{2} & \text{边界边} \end{cases}$$

2.3 顶点更新 (Vertex Point)

2.3.1 内部顶点

设顶点 v 的度 (valence) 为 n , 邻接面点均值为 F , 邻接边中点均值为 R , 原位置为 P :

$$v' = \frac{F + 2R + (n - 3) \cdot P}{n}$$

其中:

$$F = \frac{1}{n} \sum_{i=1}^n f_{p_i}, \quad R = \frac{1}{n} \sum_{i=1}^n \frac{v + v_i}{2}$$

2.3.2 边界顶点

设边界顶点 v 沿边界线的两个邻居为 $v_{\text{prev}}, v_{\text{next}}$:

$$v' = \frac{v_{\text{prev}} + 6v + v_{\text{next}}}{8}$$

2.4 面分裂

每个原始 k 边形面 (顶点 $v_0, v_1, \dots, v_{k-1}$, CCW) 分裂为 k 个四边形。设 ep_i 为边 (v_i, v_{i+1}) 的边点, fp 为该面的面点。第 i 个四边形为:

$$Q_i = (v_i, \text{ep}_i, \text{fp}, \text{ep}_{i-1})$$

每个四边形沿对角线 $v_i \rightarrow \text{fp}$ 三角化为 2 个三角形:

$$T_{i,1} = (v_i, \text{ep}_i, \text{fp}), \quad T_{i,2} = (v_i, \text{fp}, \text{ep}_{i-1})$$

2.5 规模变化

设原始网格有 V 顶点、 E 边、 F 面, 各面边数为 k_i :

$$V' = V + E + F \quad (\text{原顶点} + \text{边点} + \text{面点})$$

$$F' = 2 \sum_{i=1}^F k_i \quad (\text{每 } k_i\text{-gon 产生 } 2k_i \text{ 三角形})$$

对于纯四边形网格: $F' = 8F$; 对于纯三角网格: $F' = 6F$ 。

3 数据结构

3.1 顶点布局

输出网格的顶点数组布局:

区间	内容	数量
$[0, V)$	更新后的原始顶点	V
$[V, V + E)$	边点	E
$[V + E, V + E + F)$	面点	F

3.2 边键去重

无向边用 $(\min(i_0, i_1), \max(i_0, i_1))$ 表示，确保每条边只处理一次：

```

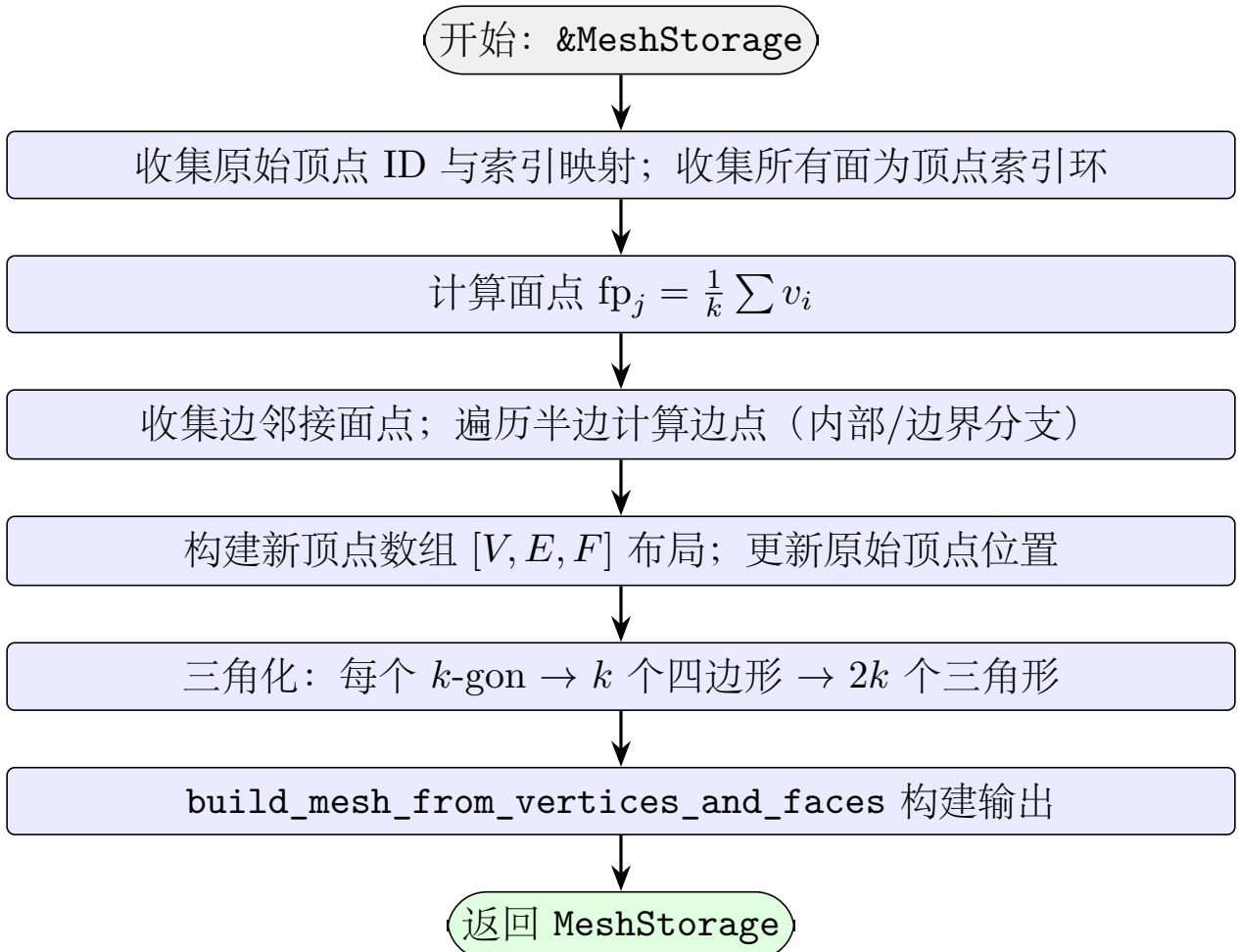
1 fn edge_key(a: u32, b: u32) -> (u32, u32) {
2     if a < b { (a, b) } else { (b, a) }
3 }

```

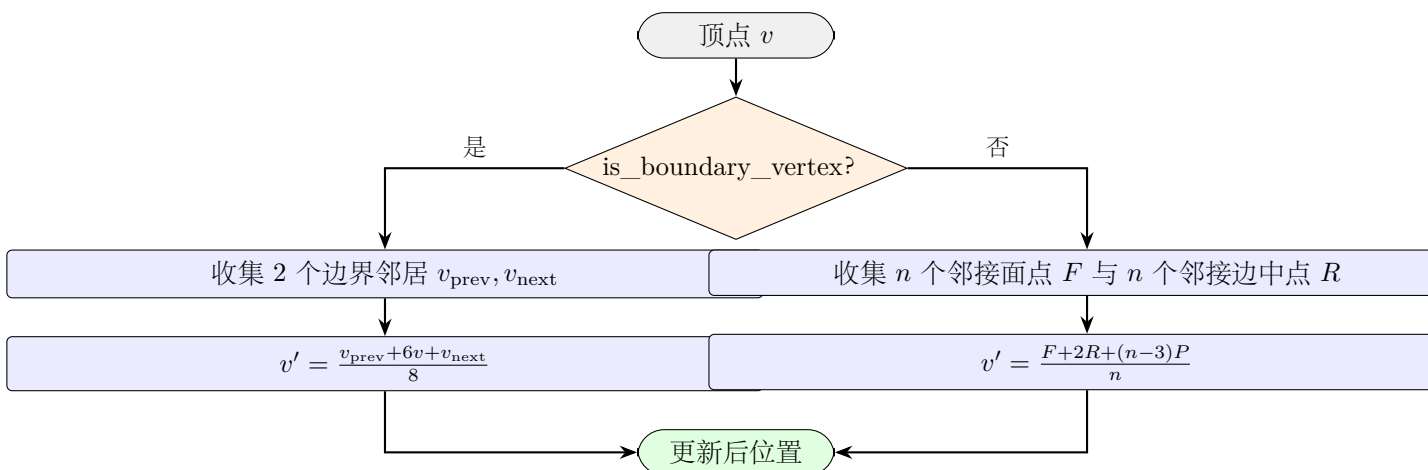
3.3 边邻接面点收集

遍历所有面的所有边，为每条边收集其邻接面点。若一条边恰好有 2 个邻接面点 $\rightarrow$ 内部边；否则（0 或 1 个） $\rightarrow$ 边界边处理。

4 流程图



5 顶点更新分支



6 与 Loop 细分的对比

特性	Loop	Catmull-Clark
输入面类型	仅三角形	任意多边形
输出面类型	三角形	四边形（三角化为 2 三角形）
面点	无	有（每面 1 个）
边点权重	$\frac{3}{8}(v_0 + v_1) + \frac{1}{8}(v_2 + v_3)$	$\frac{1}{4}(v_0 + v_1 + fp_L + fp_R)$
顶点更新	$(1 - n\beta)v + \beta \sum nb$	$\frac{F + 2R + (n-3)P}{n}$
规模变化	$V' = V + E, F' = 4F$	$V' = V + E + F, F' = 2 \sum k_i$
基础	三角 B 样条	双三次 B 样条

7 复杂度

步骤	复杂度
面点计算	$O(\sum k_i) = O(E + F)$
边点计算	$O(E)$ （每条边一次）
顶点更新	$O(V \cdot \bar{n})$ （ $\bar{n}$ = 平均度）
三角化输出	$O(\sum k_i)$
构建网格	$O(V' + F')$
总计	$O(V + E + F)$

8 边界处理

8.1 边界边点

边界边（`twin.face = None`）的边点简化为两端点中点：

$$ep_{\text{boundary}} = \frac{v_0 + v_1}{2}$$

8.2 边界顶点

边界顶点沿边界线的两个邻居 $v_{\text{prev}}, v_{\text{next}}$ 通过 `VertexRing + is_boundary_edge` 筛选。若邻居数 $\neq 2$ （退化情况），保持原位置不变。

9 退化与健壮性

- 孤立顶点 ($\text{valence}=0$): 保持原位置
- 边界顶点邻居数 $\neq 2$: 保持原位置
- 内部顶点部分边无面 ($f_count \neq n$): 保持原位置
- 面边数 < 3 : 跳过该面
- 边邻接面点数 > 2 (非流形): 退化为边界处理

10 单元测试覆盖

`src/subdiv/catmull_clark.rs` 末尾共 11 个测试:

测试名	覆盖点
<code>catmull_clark_cube_vertex_count</code>	立方体 $V=26$ ($8+12+6$)
<code>catmull_clark_cube_face_count</code>	立方体 $F=48$ ($6*4*2$)
<code>catmull_clark_cube_halfedge_count</code>	立方体 $HE=144$ ($3*48$)
<code>catmull_clark_cube_passes_validation</code>	完整拓扑校验通过
<code>catmull_clark_cube_preserves_euler_characteristic</code>	Euler $\chi = 2$ 保持
<code>catmull_clark_single_quad</code>	单四边形 $V=9, F=8$
<code>catmull_clark_triangle_input</code>	三角形输入 $V=7, F=6$
<code>catmull_clark_closed_tetrahedron</code>	闭合四面体 $V=14, F=24, \chi = 2$
<code>catmull_clark_cube_face_point_at_center</code>	面点位置在面中心
<code>catmull_clark_cube_vertex_moves_inward</code>	角点向中心移动
<code>catmull_clark_double_subdivide</code>	二次细分 $V=146$, 拓扑校验通过

11 设计取舍

- 为何输出三角化而非保留四边形? 当前 `MeshStorage` 的 `validate_topology` 要求面边界环长度 $=3$ 。三角化输出确保与现有校验工具链兼容。未来若放宽三角约束 (方案 b)，可直接输出四边形。
- 为何使用 `build_mesh_from_vertices_and_faces` 而非直接构造半边? 复用成熟的 `builder` 逻辑 (`twin/next/prev/边界环自动建立`)，避免 Catmull-Clark 模块重复实现拓扑构建代码。
- 为何边点计算需要先收集邻接面点? Catmull-Clark 边点公式依赖两侧面点。通过 `edge_face_points` `HashMap` 预收集，避免遍历半边时重复查询面点。

- 为何内部顶点公式使用边中点而非边点？标准 Catmull-Clark 公式中 R 是邻接边中点均值 $(v + v_i)/2$ ，而非邻接边点均值。中点仅依赖原始顶点位置，与边点（依赖面点）解耦，保证计算顺序正确。